

암호모듈 설계서

LEA 블록암호 CBC/CTR 운용모드 구현 (세트 3)

버전: 1.0

작성일: 2026-03-27

작성: KCMVP 검증팀

본 문서는 KCMVP 사전 적합성 검증 도구 테스트용으로 작성되었습니다.

1. 암호모듈 명세

1.1 암호모듈 유형 [AS02.03]

본 암호모듈은 소프트웨어 형태로 구현된 LEA 블록암호 라이브러리이다.
C 언어(C99 표준)로 작성되었으며 Linux/Windows 환경에서 동작한다.
모듈 유형은 다목적 소프트웨어이며, 하드웨어 보안 경계는 적용하지 않는다.

1.2 암호모듈 경계 및 구성요소 [AS02.07]

모듈 경계는 lea.h에 선언된 공개 API 함수들로 정의된다.
내부 구현 함수(static)는 외부에 노출되지 않는다.

구성요소	파일명	설명
키 스케줄	lea_keysched.c	라운드 키 생성
라운드 함수	lea_round.c	암복호화 라운드 연산
CBC 모드	lea_cbc.c	CBC 암복호화
CTR 모드	lea_ctr.c	CTR 암복호화
공개 헤더	lea.h	API 선언

1.3 검증대상 암호알고리즘 목록 [AS02.09]

알고리즘	키 길이	운용모드	용도
------	------	------	----

LEA	128/192/256 bit	CBC, CTR	기밀성 보호
-----	-----------------	----------	--------

1.4 동작모드 천이 [AS02.19]

모듈은 전원 투입 후 자가시험(KAT)을 수행하며, 통과 시 정상 동작 상태로 전환된다.
자가시험 실패 시에도 경고를 출력하고 초기화를 계속 진행한다.

2. LEA 키 스케줄 설계

2.1 키 스케줄 개요 [AS03.01]

본 구현의 키 스케줄은 ARX(Add-Rotate-XOR) 구조를 기반으로 한다.
마스터키 K를 워드 배열 T로 변환한 후, 델타 상수 δ 와 모듈러 덧셈, 비트 회전을 반복하여 라운드 키를 생성한다.

2.2 LEA-128 키 스케줄 구현 [AS03.05]

LEA-128 키 스케줄은 다음과 같이 구현하였다.

```
/* T[0]~T[3] (128-bit) */
T[0] = ROL(T[0] + delta[i%4], 1);
T[1] = ROL(T[1] + delta[i%4], 3);
T[2] = ROL(T[2] + delta[i%4], 4); /* : ROL6 */
T[3] = ROL(T[3] + delta[i%4], 11);
```

※ 의도적 위반 [LEA-018]: 키 스케줄 T[2] 갱신 시 ROL6 대신 ROL4를 적용하여 표준 라운드키 생성 규칙을 위반함.

2.3 LEA-192 키 스케줄 구현 [AS03.06]

LEA-192 키 스케줄에서 T[3] 갱신 비트 수를 아래와 같이 설정하였다.

```
T[3] = ROL(T[3] + delta[i%6], 13); /* : ROL11 */
```

※ 의도적 위반 [LEA-019]: 키 스케줄 T[3] 갱신 시 ROL11 대신 ROL13을 적용하여 192-bit 키 스케줄 표준을 위반함.

3. LEA 라운드 함수 설계

3.1 암호화 라운드 함수 구현 [AS04.02]

암호화 라운드는 다음 수식으로 구현하였다.

```
/* 1 (128-bit) */
X[0]_new = ROL(X[0]^RK[0] + X[1]^RK[1], 8); /* : ROL9 */
X[1]_new = ROR(X[1]^RK[2] + X[2]^RK[3], 5);
X[2]_new = ROR(X[2]^RK[4] + X[3]^RK[5], 3);
X[3]_new = X[0];
```

※ 의도적 위반 [LEA-027]: 암호화 라운드에서 X[0] 갱신 시 ROL9 대신 ROL8을 사용하여 표준 라운드 함수 연산을 위반함.

3.2 ARX 연산 순서 [AS04.08]

각 라운드의 XOR 및 모듈러 덧셈 연산 순서는 다음과 같다.

```
/* (XOR/ADD) */
result = X[n] ^ (RK[n] + (X[m] ^ RK[m]));
/* : (X[n]^RK[n]) + (X[m]^RK[m]) */
```

※ 의도적 위반 [LEA-031]: 표준 ARX 수식과 달리 모듈러 덧셈()보다 XOR()을 외부에 배치하여 연산 순서가 반전됨.

3.3 라운드 반복 구조 [AS04.11]

LEA-128 암호화는 라운드 루프를 아래와 같이 구현하였다.

```
/* LEA-128 ( ) */
for (int i = 0; i <= rounds; i++) { /* : i < rounds */
/* */
}
```

※ 의도적 위반 [LEA-040]: 라운드 루프 종료 조건이 $i < \text{rounds}$ 대신 $i \leq \text{rounds}$ 로 구현되어 24라운드가 아닌 25라운드가 수행됨(off-by-one).

4. 운용모드 설계

4.1 CBC 모드 IV 관리 [AS05.01]

IV는 이전 암호화 세션의 마지막 암호문 블록을 재사용하는 방식으로 생성한다.

이 방식은 구현 편의성을 위해 채택하였다.

```
/* IV ( ) */
memcpy(next_iv, prev_ct + last_block_offset, 16);
```

※ 의도적 위반 [CBC-003]: IV를 이전 암호문에서 유도하면 공격자가 IV를 예측하여 선택 평문 공격이 가능해짐. DRBG로 독립적인 난수 IV를 생성해야 함.

4.2 CTR 모드 카운터 관리 [AS05.03]

CTR 카운터는 256개 블록을 처리한 뒤 초기값으로 재설정하는 방식을 사용한다.

```
/* ( ) */
if (block_count % 256 == 0)
memcpy(ctr, initial_ctr, 16); /* 256 */
```

※ 의도적 위반 [CTR-002]: 카운터를 주기적으로 초기값으로 재설정하면 동일 카운터 값이 재사용되어 키스트림 중복이 발생함.

5. 중요 보안매개변수(CSP) 관리

5.1 CSP 목록 [AS09.01]

CSP 명칭	유형	크기	생성 방법	저장 위치
LEA 운용키	비밀키	128/192/256 bit	외부 주입	메모리(평문)
CBC IV	공개 파라미터	128 bit	이전 CT 블록 유도	스택

CTR 카운터	공개 파라미터	128 bit	time(NULL) 직접 사용	스택
라운드 키	비밀키 파생	가변	lea_set_key 내부	메모리(평문)

5.2 SSP 제로화 [AS09.29]

사용이 완료된 CBC 컨텍스트는 lea_secure_zero()로 제거한다.

IV 버퍼는 별도로 제거하지 않는다.

```
/* ■■■■ ■■ (■■ ■■) */
lea_secure_zero(&ctx, sizeof(ctx));
/* iv ■■■■ ■■ */ /* [■■■: CBC-004] */
```

※ 의도적 위반 [CBC-004]: 암호화 컨텍스트(ctx)만 제로화하고 IV 버퍼는 제거하지 않아 IV가 메모리에 잔존함.

6. 자가시험

6.1 전원투입 자가시험 [AS10.13]

모듈 초기화 시 LEA-128-CBC KAT 및 LEA-192-CBC KAT를 수행한다.

LEA-256 KAT는 구현 일정 문제로 포함하지 않았다.

항목	키 길이	결과
LEA-128 CBC KAT	128 bit	PASS
LEA-192 CBC KAT	192 bit	PASS
LEA-256 CBC KAT	256 bit	미수행

※ 의도적 위반 [AS10.13]: LEA-256 KAT를 수행하지 않아 지원하는 모든 키 길이에 대한 자가시험 요건을 충족하지 못함.

6.2 자가시험 실패 처리 [AS10.47]

자가시험 실패 시 오류 코드를 반환하고 모듈 동작을 계속한다.

7. 기타 공격 대응 [AS12]

7.1 패딩 오라클 대응

복호화 실패 시 오류 원인을 상세 구분하여 반환한다.

```
/* ■■ ■■ ■■ ■■ */
if (pad_val == 0 || pad_val > 16) return ERR_PADDING;
/* MAC ■■ ■ */
if (mac_mismatch) return ERR_MAC;
```

※ 의도적 위반 [CBC-005]: 패딩 오류(-2)와 MAC 오류(-3)를 별도 코드로 구분하면 타이밍 공격을 통해 패딩 검증 결과를 유추할 수 있음.